Information Sciences 180 (2010) 3932–3954

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Validating a size measure for effort estimation in model-driven Web development

Silvia Abrahão [a,*], Jaime Gómez [b,1], Emilio Insfran [a]

[a] ISSI Research Group, Department of Information Systems and Computation, Universidad Politécnica de Valencia, Camino de Vera, s/n, 46022 Valencia, Spain
[b] Department of Information Systems and Languages, University of Alicante, Campus de San Vicente del Raspeig, Apartado 99, 03080 Alicante, Spain

ARTICLE INFO

Article history:
Received 10 August 2009
Received in revised form 16 May 2010
Accepted 28 May 2010

Keywords:
Model-driven development
Web engineering
Functional size measurement
Effort estimation
Measure validation

ABSTRACT

Web development is moving towards model-driven processes whose goal is the development of Web applications at a higher level of abstraction based on models and model transformations. This brings new opportunities to the Web project manager to make early estimates of the size and the effort required to produce Web applications based on their conceptual models. In the last few years, several studies for size and effort estimation have been performed. However, there are no studies regarding effort estimation in model-driven Web development. In this paper, we present the validation of a model-based size measure (OO-HFP) for Web effort estimation in the context of a model-driven Web development method. The validation is performed by comparing the prediction accuracy that OO-HFP provides with the accuracy provided by the standard function point analysis (FPA) method. The results of the study (using industrial data gathered from 31 Web projects) show that the effort estimates obtained for projects that are sized using OO-HFP are more accurate than the effort estimates obtained using the standard FPA method. This suggests that by following a model-driven development approach, the size measure obtained at the conceptual model of a Web application can be considered a suitable predictor of effort.

© 2010 Elsevier Inc. All rights reserved.

1. Introduction

Web development is significantly different from traditional software development [44]. The traditional balance between the project tradeoffs of time, cost, and quality has changed. The nature of Web development forces project managers to focus primarily on the time variable in order to achieve the required short cycle times. Because of this imbalance, several problems such as exceeding budgets and bad product quality have been reported in Web projects [16].

To address these issues, model-driven development approaches [43] seem to be very promising since Web development can be viewed as a process of transforming models into successive ones until they can be executed in a target platform. Models can represent different abstraction levels of a Web application, starting from a platform-independent model to a platform-specific model, and finally to a code-level model. Over the last few years, a significant number of Web development methods that follow a model-driven approach have been proposed (e.g., OOHDM [50], WebML [10], OO-H [22], W2000 [7]). Although most of these methods were developed in academic contexts, they have also been successfully applied in several industrial settings, including, e.g., ACER in Europe, the Middle East, Africa, and the Pan American regions, as well as several banking institutions across Spain [40].

* Corresponding author. Tel.: +34 96 3879351; fax: +34 96 3877359.
E-mail addresses: sabrahao@dsic.upv.es (S. Abrahão), jaime.gomez@ua.es (J. Gómez), einsfran@dsic.upv.es (E. Insfran).
[1] Tel.: +34 96 5903772x3306; fax: +34 96 5909326.

0020-0255/$ - see front matter © 2010 Elsevier Inc. All rights reserved.
doi:10.1016/j.ins.2010.05.031

Adopting such methods, however, poses new challenges to the Web project manager, especially with respect to resource estimation and project planning. A fundamental problem in this context is the size and effort estimation of the future Web application based on its conceptual model. As suggested in [48], analysis objects (i.e., number of methods) are significantly correlated with the size of a software product. Other approaches adapted standard functional size methods (e.g., COSMIC, FPA) for sizing Web sites and applications [9,11,15]. However, the main limitation of these approaches are the following: (i) they rely on design and/or implementation decisions; (ii) they are not automated [9,11,23,44], which limits their use and adoption; and (iii) their conformance to standards relies heavily on the expertise of the measurer.

In order to deal with these issues, in [2], we proposed a measurement procedure for Web applications (object-oriented hypermedia function points (OO-HFP)), which is compliant with the FPA method. A measurement procedure is defined as a "set of operations, described specifically, used in the performance of particular measurements according to a given method of measurement" [27]. The aim of OO-HFP is to size Web applications that are developed following a model-driven development approach using the object-oriented hypermedia (OO-H) method [22].

The main objective of this paper is to validate the OO-HFP for effort estimation in model-driven Web development by comparing the prediction accuracy that it provides with the accuracy provided by the standard FPA method. The reason for validating OO-HFP as a Web effort predictor is the need for an accurate effort predictor at the conceptual model level of a Web application. Although some recent studies [17,49,37] have compared the accuracy of some Web size measures as effort predictors, none of these measures have been validated in the context of a model-driven Web development process. Thus, the main contribution of this work is to present an empirical study that validates the usefulness of a model-based size measure for effort estimation in a specific model-driven development process. We believe that this paradigm gives the estimator a new and earlier chance for estimating size and effort in an accurate way, since modeling is in fact, programming at a higher level of abstraction.

The reason for choosing the standard FPA is that it has been widely used in industry. The International Software Benchmarking Standards Group [24] provides a large database of industrial software and Web projects where this size measure was employed. Since OO-HFP was proposed in order to facilitate the size estimation of Web applications when applying the FPA method, we can verify whether or not OO-HFP is more accurate than FPA for estimating the size of Web applications. These two measures are compared using data gathered from 31 Web projects developed by a Web development company. We used both size measures as effort predictors in regression models.

This paper is organized as follows. In Section 2, we present an overview of related work on functional size measures for Web applications with special emphasis on their use for Web effort estimation. In Section 3, we briefly introduce the reader to the OO-H development process. In Section 4, we present an overview of the OO-HFP, which is followed in Section 5 by its application to a case study. In Section 6, we show the validation of OO-H as a Web effort predictor. Finally, in Section 7, we present conclusions and outline future work.

2. Related work

In the last few years, several Web measures to estimate the size of Web applications have been proposed. Some representative measures can be found in [38,39,32,15,6]. However, to date, there are no widely accepted Web measures in industry. Among these measures, we believe that a functional size measure could provide a good solution to the problem of sizing Web applications since: (i) it is based on the functionality that will be provided to the users and not on the artifacts that are produced when the Web application is delivered (e.g., number of Web pages, multimedia files); (ii) it can be measured in early stages of the Web development process; (iii) it is based on standards that define the concept of size and the requirements for software sizing (e.g., [25,26]); and (iv) it is widely accepted and used in industry [24]. In the following, we review existing functional size measures for Web applications as well as their use in Web effort estimation.

2.1. Functional size measures for Web applications

Most Web development methods break up the application's design in three models: content, navigation and presentation. The content model defines the application's underlying structure and behavior. Content is commonly represented using UML class diagrams or entity-relationship (ER) diagrams. The navigation model builds the nodes and links from content model's classes and relationships, and finally, the presentation model defines the abstract graphical interface from each node, thus defining the appearance of the Web pages by means of abstract user interface elements.

The functionality represented in content models can be estimated by applying traditional size measures based on UML diagrams (e.g., class points [14], Uemura et al. [52], object-oriented function points [5]). These measures were conceived to estimate the size of object-oriented systems based on design documentation. Therefore, they can be used to estimate the static and behavioral views of Web applications. For instance, the class point method is specifically focused on classes. The method considered four different types of classes (e.g., classes that offer functionality for data storage and retrieval). The complexity of each class is derived from the information about the local methods in the class, the interaction of the class with the rest of the system, and, when available, the attributes [14]. However, these alternative size measures do not provide rules for measuring the specific properties of Web applications commonly represented in navigation models and abstract user interface models that are built upon content models.

This change in development practices makes the existing size measures for object-oriented systems not sufficient to properly estimate the size of Web artifacts. Indeed, to cope with size estimation in Web development, several functional size measurement procedures have been proposed. The first attempt to apply functional size measures to Web artifacts were the guidelines proposed by IFPUG [23]. However, some practitioners [51] stated that these guidelines did not resolve many of the counting issues faced when sizing Web applications. The practitioners provided interpretations for the IFPUG guidelines and explained how FPA can be applied to Web artifacts. Rollo [47] also discussed some of the difficulties when measuring Web applications with FPA, such as the difficulties encountered in identifying the system's boundary and its logical files. Rollo [47] argues that COSMIC is more flexible for counting Web applications. However, no evidence of this comparison was provided.

Widely adopted FSM methods such as FPA and COSMIC do, in fact, have some relevant limitations. The first problem is that the application of the rules requires human interpretation (e.g., it is not always clear how every element of the requirements should be classified and counted), consequently, an expert on applying the method is required (e.g., a certified function point specialist). The second problem is that none of the ISO-standardized FSM methods have been designed to take the particular features of Web applications into account.

To address these limitations, 10 size measurement procedures for Web applications have been proposed in the last few years. Of these procedures, four were proposed by practitioners (IFPUG guidelines for Web applications [23], Web-points [11], Web objects [46], internet points [13]). The remaining six were proposed by researchers (OOmFPWeb [3,4], data web points [41], Cândido and Sanches [9], Costagliola et al. [15], Fraternali et al. [20] and OO-HFP [2]). These measurement approaches are summarized and listed chronologically in Table 1.

On the other hand, the size measurement procedures specifically proposed for Web applications (see Table 1) suffer from some drawbacks. With the exception of Web-points [11], OOmFPWeb [4], and Fraternali et al. [20], they do not present a well-defined set of mapping and measurement steps to apply functional size measurement to Web artifacts. Also, some of the approaches (e.g., Web objects [46], Web-points [11], and internet points [13]) cannot be used in the early stages of the Web development lifecycle since they rely on decisions regarding implementation languages and technologies. Furthermore, some approaches like Web objects [46], Web-points [11], and data Web-points [41] do not use an ISO-standardized measurement method, but rather define new methods of sizing. One disadvantage of this is that the size measured with these approaches is not comparable with the size standards that are currently accepted in practice.

All the proposals provide rules to size the static dimension of a Web application (i.e., the structural aspects commonly represented in a data/object model). In [15], a preliminary mapping between Conallen's UML extension and COSMIC is proposed. The procedure is based on counting data movements (to and from Web servers) and turns out to be suitable for capturing the specific aspects of dynamic Web applications. Some approaches also provide measurement rules for measuring the navigation and/or presentation viewpoint. This is the case of Web objects that take into account the number of links (xml, html and query language lines) as part of the size of a Web application.

However, only OOmFPWeb [4], Fraternali et al. [20], and OO-HFP [2] provide measurement rules to size all the dimensions of a Web application (i.e., data, behavior, navigation, and presentation). In fact, these proposals are somewhat similar. OOmFPWeb [4] was proposed as a size measurement procedure for Web applications as a mapping between the OOWS conceptual modeling primitives [45] and the FPA concepts, whereas Fraternali et al. [20] and OO-HFP applied the same strategy but using the WebML and OO-H conceptual modeling primitives, respectively. Of these methods, only WebML and OO-H have been employed in industrial settings. This suggests that only the size measures defined for these methods can be properly validated in practice given the availability of industrial project data.

Table 1
Comparison between FSM methods and procedures for Web applications.

Proposal	Measurement method	Input artifact	Dimension measured[a]	Automated support	Validation
IFPUG guidelines [22]	IFPUG	Final Web application	D	No	No
Web-points [11]	–	Final Web application	D, N	No	No
Web objects [45]	–	Final Web application	D, N	No	Validation of the method as Web effort predictor [48,17,19]
Internet points [13]	IFPUG	Final Web application	D, P	Cost Xpert	No
OOmFPWeb [4]	IFPUG 4.1.1	Object-oriented conceptual models	D, B, N, P	No	Validation of the procedures' reproducibility, perceived ease of use, perceived usefulness and intention of use [3]
Data Web-points [40]	–	Data model	D	No	Validation of the method's accuracy [40]
Cândido and Sanches [9]	NESMA	Final Web application	D	Yes	Validation of the procedure's accuracy [9]
Costagliola et al. [15]	COSMIC	Design models	D, B, N	No	Validation of the procedure as Web effort predictor [15]
Fraternali et al. [20]	IFPUG 4.1.1	Data-oriented conceptual models	D, B, N, P	WebRatio plug-in	Comparison of the manual and automated measurement results [20]
OO-HFP [2]	IFPUG 4.1.1	Object-oriented conceptual models	D, B, N, P	VisualWade plug-in	Validation of the procedure as Web effort predictor [2]

'–' Means that the proposal defines a new method of functional size measurement.
[a] D = Data, B = Behavior, N = Navigation, P = Presentation.

Apart from these problems, few evaluations of the validity of the proposed FSM alternatives for Web applications have been documented in the literature. The data web points procedure [41] was tested using graduate and undergraduate students in computer science who were taking a regular course in software engineering. The course instructor and two industry experts evaluated the quality of the size measures obtained with data web points [41] in a subjective manner. The size estimation was evaluated as: good, if it was very similar to the expert's opinion; medium, if the estimation and the expert's opinion were close; and poor, if they were not similar. The results for 22 Web projects were reported: 15 projects had good size estimation; five projects had medium size estimation (two of these projects needed minor adjustments, and three of them were incomplete); and two projects had poor size estimation (both were incomplete).

In [9], 20 Web applications were counted using the IFPUG FPA method, the NESMA FPA method (both the estimative and indicative counting procedures), and the simplified method proposed in that work. This simplified method is based on assigning a low complexity to all data and transactional functions. The results of the study found that the estimates using the simplified method were close to the ones using the IFPUG FPA method (a difference of 4% was observed).

OOmFPWeb has been empirically validated using an evaluation framework for FSM methods [1,3]. The results of a family of experiments with students showed that OOmFPWeb is efficient when compared to current industry practices. The results also showed that the procedure produces reproducible size measurements and that it is perceived as easy to use and useful by the study participants (who also expressed their intention to use this procedure in the future). However, OOmFPWeb could not be validated for Web effort estimation given the lack of industrial Web projects obtained with the OOWS methodology.

In [20], a comparison between manual and automatic counting using two different algorithms (pattern-based and duplicate pruning) for four different Web applications was reported. The results for the automatic counting using the duplicate pruning algorithm differed from the manual counting by a maximum of 11%. This difference was detected in the duplicate pruning algorithm due to its inability to identify some complex design patterns. However, Fraternali et al.'s procedure [7] has not yet been validated for Web effort estimation purposes.

2.2. Web effort estimation

With regard to Web effort estimation, several studies have been reported in the literature [39,49,15,17,19]. A few of these studies compared the effectiveness of some of the above-mentioned measures with other general-purpose measures. For instance, in the Ruhe et al. [49] study, the Web objects method was applied and validated for the first time. In this work, effort estimation models based on Web objects and FPA were compared using ordinary least square regression. The authors of that study found that the performance of Web objects was superior to the FPA standard method.

In another study, Costagliola et al. [15] reported on an empirical analysis about the effectiveness of COSMIC to predict Web application development effort. The study used 44 Web applications developed by students. The results of the study suggest that the counting of data movements can be useful for estimating the development effort of dynamic Web applications.

Similarly, Di Martino et al. [17] compared the effectiveness of four Web size measures (i.e., Tukutuku measures, Web objects, length measures, and functional measures) for predicting Web application development effort. The results of the study (using data from 15 Web projects developed by a software company) showed that all the measures provided good effort predictions. In another study, Ferrucci et al. [19] compared COSMIC and Web Objects in order to verify their effectiveness as indicators of development effort. The results of a study including a dataset with 15 Web applications reveal that both Web objects and COSMIC are good indicators of the development effort.

However, despite their promising results, the above studies are inconclusive with regard to identifying the best effort predictor in Web development. In other words, there is no single effort model that can be applied in all situations or company circumstances. Furthermore, there is a lack of studies on effort estimation for model-driven Web development. To the best of our knowledge, there is only one study performed in this context [2]. In this study, we presented a preliminary evaluation of OO-HFP as a Web effort predictor by comparing the prediction accuracy that it provides with the accuracy provided by another set of validated Web size measures (the Tukutuku measures [37]). The results of that study, based on the analysis of 12 industrial Web projects, showed that the effort estimates obtained for projects that were sized using OO-HFP were similar to those obtained using the Tukutuku measures, suggesting that OO-HFP is a suitable Web effort predictor.

However, more experimentation is needed to establish a body of knowledge in the field. These experimentations should be performed to confirm the previous empirical results. Here, we apply and validate the OO-HFP approach for effort estimation purposes in the context of a Spanish Web development company that employs a model-driven Web development method (OO-H).

3. The object-oriented hypermedia method

The object-oriented hypermedia (OO-H) method [22] provides the semantics and notation for developing Web applications. The main feature of this method is its model-driven and transformation-based approach. Following, we introduce the OO-H development process and its metamodel.

3.1. The OO-H development process

According to the model-driven architecture approach defined by the object management group [43], the OO-H development process (see Fig. 1a) starts by defining a *requirements model* (computation-independent model, CIM). Skeletons of platform-independent models (PIMs) can be obtained from these requirements and then manually completed to represent the main dimensions of a Web application: *content* (UML class diagram); *navigation* (navigation access diagram, NAD); and *presentation* (abstract presentation diagram, APD).

OO-H associates a different NAD to each type of user. The NAD captures the navigation paths and services that the user can activate when working with the Web interface. From the UML class diagram and the NADs, a Web interface can be generated without further work because OO-H method provides a set of default values for the main presentation features. This allows the designer to shorten the time needed to develop Web applications (to get more information about the NAD and the OO-H method, interested readers are referred to [22]).

However, once an agreement between the designer and the stakeholder has been reached, the designer will most probably need to modify this default Web interface in order to improve both its appearance and usability features. In order to do this, OO-H method defines another diagram: the abstract presentation diagram (APD). This diagram is obtained by merging the class and navigation diagrams. Finally, platform-specific models (PSMs) are automatically obtained from the APD, which can be manually modified in order to adjust to the user interface requirements. Source code can then be generated from the APD to obtain the Web application. The aim of this model-driven development process is to automate the definition and transformation of models based on a set of predefined transformation rules that are specific for each target platform. The VisualWADE tool [53] automates the entire OO-H development process.

3.2. The OO-H metamodel

This section explains the main concepts of the OO-H metamodel [22] that are used to represent navigational properties (see Fig. 2). The navigation model of a Web application is defined by a set of navigational maps. Each *navigational map* structures the navigational view of the system for a specific type of *user* and has a unique *entry point* that indicates the starting point of the navigation process. A navigational map is made up of a set of *navigational elements* that can be specialized as navigational nodes and/or navigational links. A *navigational node* represents a view over the class diagram. A navigational node can be a navigational target, a navigational class, or a collection.

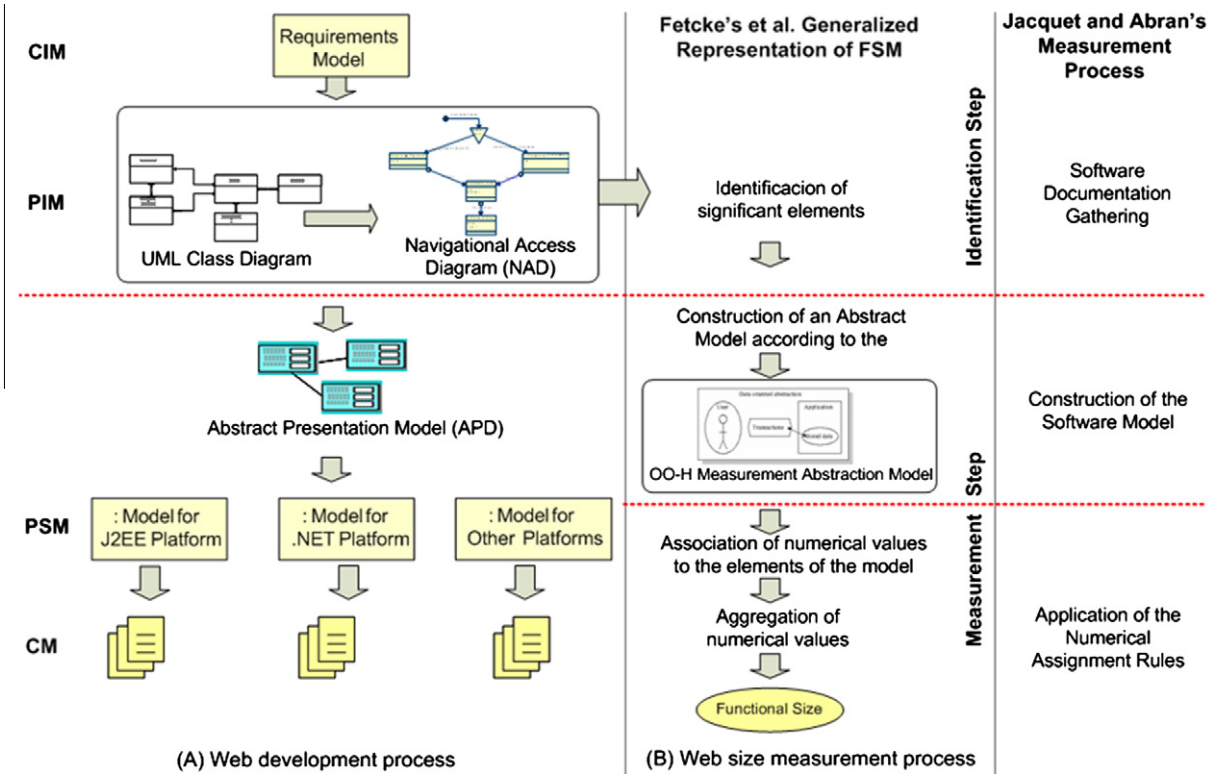

Fig. 1. Overview of OO-H and OO-HFP. (A) Shows the OO-H development process and (B) shows the steps of the OO-HFP measurement process.

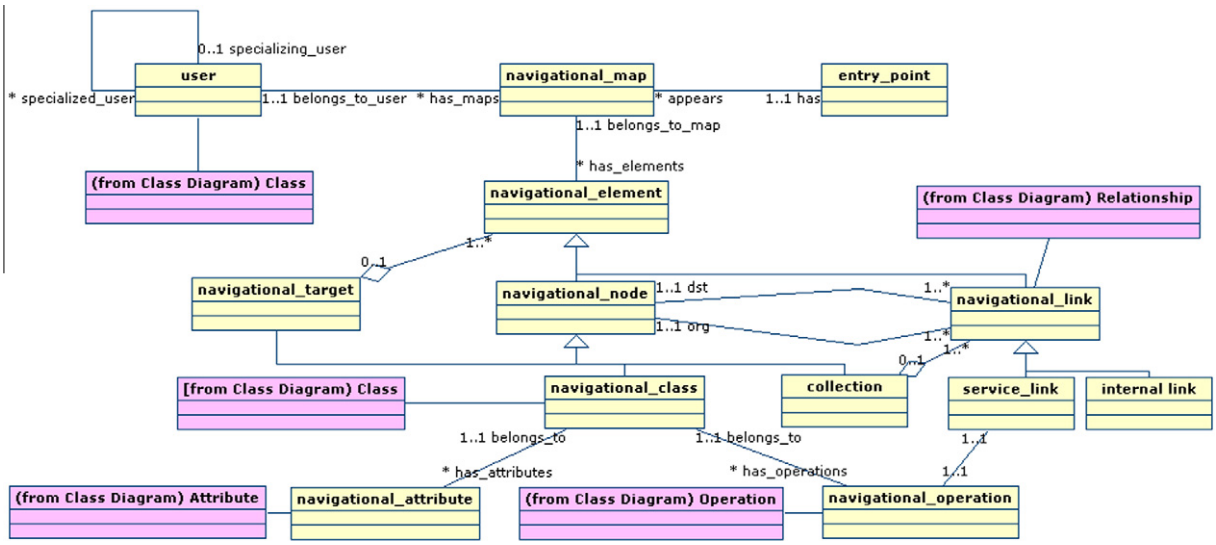

Fig. 2. An excerpt of the OO-H metamodel.

A *navigational target* groups some elements of the model (i.e., navigational classes, navigational links, collections) that collaborate in the coverage of a user navigational requirement. A *navigational class* represents a view over a set of attributes (*navigational_attribute*) and operations (*navigational_operation*) of a class from the class diagram. A *collection* is a hierarchical structure that groups a set of navigational links. A *navigational link* defines valid node attainability inside the navigational map. Navigational links define the navigation paths that the user can follow through the user interface. Each navigational link has an origin (*org*) node and its corresponding destination (*dst*) node. There are two types of links: *internal links* to define the navigation path inside a navigational map, and *service links* to show the services available to the user associated to that navigational map.

In addition, OO-H allows the use of the object constraint language (OCL) to define constraints over the class diagram and the NAD. In the NAD, such constraints are expressed by means of *filters* that are defined upon links (internal or service).

4. OO-HFP overview

The OO-HFP [2] is a FSM procedure that conforms the standard IFPUG FPA method [25]. It takes into account the domain (UML class model) and the navigation (NAD) views as input to calculate the size. These views contain all the elements that contribute to the functional size of a Web application.

Given the conformity of this procedure to FPA, these elements are, in essence, the same as the ones described in the FPA metamodel; they basically distinguish five function types: internal logical files (ILF), external interface files (EIF), external inputs (EI), external outputs (EO), and external inquiries (EQ). These elements are identified based on a collection of *mapping rules* (see Section 4.1). OO-HFP also provides another set of rules, the *numerical assignment rules* (see Section 4.2), which are used to identify the elements that add to the complexity of the identified function types. Finally, the IFPUG tables are used to rate the complexity of the functions, to assign weights to the elements of the model, and to aggregate the assigned values into an overall functional size value for the Web application. The calculation of the OO-HFP size measure was automated in VisualWADE [53], which is a tool that supports the OO-H development process. This avoids the ambiguity of interpreting the FPA counting rules and the need for special training to count function points in an accurate and repeatable way.

4.1. The mapping rules

The mapping rules help to identify the elements in an OO-H conceptual model that contribute to the size of the Web application. These rules (see Table 2) are defined as a mapping between the concepts of the FPA metamodel onto the concepts in the UML/OO-H metamodel [42,22]. The use of metamodels in software measurement has shown to be effective [21].

First, the rules to establish the counting scope and the boundary of the Web application are applied (Rules 1–4). The *counting scope* defines the functionality which will be included in a particular function point count. It defines a (sub)set of the software being sized. To properly define the counting scope, the user view should be previously established. In fact, the user view defines how the user sees the application. This is an important activity before measurement because in a Web environment this view can vary. For instance, a Web portal offers a variety of Internet services from a single location. It can be viewed as different applications in the same location. Thus, the user can view the portal as a single application or as separate applications.

Table 2
Mapping rules.

FPA Metamodel	UML/OO-H Metamodels
Counting scope	Rule 1. The scope corresponds to an OO-H conceptual model including a UML class diagram and a navigation access diagram (NAD)
Boundary	Rule 2. Accept each class with the "actor" stereotype as a user of the application
	Rule 3. Accept each class with the "legacy" stereotype as an external application. This class contains the expected object behavior or the connection with existing applications
	Rule 4. The boundary corresponds to an imaginary line traced in the UML class diagram. The "actor" and "legacy" classes are considered to be outside the boundary, whereas the other classes are considered to be inside the boundary
ILF	Rule 5. Accept each aggregation hierarchy as an ILF, since aggregation represents "is part of" relationships
	Rule 6 Accept each composition hierarchy as an ILF. Composition is stronger than aggregation since the whole and the parts have coincident lifetimes
	Rule 7. Accept each class of the class diagram that does not participate in an aggregation or composition hierarchy as an ILF
EIF	Rule 8: Accept each class with the "legacy" stereotype of the class diagram as an EIF
EI	Rule 9: Accept each method of a class in the class diagram as an EI
EQ	Rule 10: Accept each navigational target of a NAD (and each abstract page derived from it) as an EQ
EO	Rule 11: Accept each navigational target of a NAD (and each abstract page derived from it) as an EO. It must contain at least a class with a derived attribute

Therefore, how can a correct counting scope be defined? This is one of the problems to be deal with when designing functional size methods for Web applications. In our approach, this problem is solved since we measure conceptual models that were previously defined from the user point of view. Using the OO-H conceptual model primitives, one or more applications can be designed in accordance with the business goal and the requirements of the Web application.

In OO-H, a navigation access diagram (NAD) is built to represent the information and the functionality that a type of user can access as well as the valid paths that she/he can go through the Web application. Therefore, a navigation access diagram represents the view that a particular type of user has of the Web application. As a result, the counting scope includes the UML class diagram as well as the navigation access diagrams that are built for each type of user.

The *boundary* indicates the border between the software being measured and the user. In OO-H, each class of the UML class diagram with the "actor" stereotype is considered to be a user type of the Web application whereas each class with the "legacy" stereotype is considered to be an external application. Therefore, the boundary corresponds to an imaginary line traced in the UML class diagram. The "actor" and "legacy" classes are considered to be outside the boundary, whereas the other classes are considered to be inside the boundary.

Once the counting scope and the boundary are established, the data (ILF and EIF) and transactional (EI, EO, and EQ) functions are identified. In FPA, an ILF is defined as *a user identifiable group of logically related data or control information maintained within the boundary of the system*. In OO-H, this corresponds to a *class* that encapsulates a set of data items (attributes) representing the state of the objects in each class. Aggregation and composition hierarchies can also be considered an ILF since they represent a unique group of logically related data (Rules 5–7).

An EIF is *a user identifiable group of logically related data or control information referenced by the system, but maintained within the boundary of another system*. In OO-H, this is a special type of *class with the "legacy" stereotype* which allows the connection with existing external applications (Rule 8).

An EI *is an elementary process that manipulates data or control information that comes from outside the system boundary*. Its goal is to maintain one or more ILFs and/or to alter the behavior of the system. In OO-H, this corresponds to a *method* of a class, since it always changes the state of the objects of the class. However, a method must have *public* visibility and must be identified only once (in the declared class), even if it is inherited (Rule 9). Methods in a class with public visibility can be seen and used from outside the system boundary. The parameters of these methods specify the data that cross the system boundary.

An EQ *is an elementary process that sends data or control information outside the system boundary*. Its goal is to present information to a user through the retrieval of data. In OO-H, this corresponds to a *navigational target* defined in the NAD (and each abstract page derived from it) since it may describe different kinds of user interactions with the Web application, where data generated within the system is presented to the user.

A navigational target may derive one or several *abstract pages* depending on the value of the property 'effect' of a navigational link. The value *source* (a light arrow) indicates that the information specified in the navigational target will be presented in the current abstract page; the value *target* (a dark arrow) indicates that the information will be presented in a different abstract page (Rule 10). For an example of these types of navigational links see Fig. 4.

Finally, an EO has the same definition as an EQ, but its processing logic must contain at least one mathematical formula or calculation, or it has to create derived data. In OO-H, this also corresponds to the concept of *navigational target*. However, at least a class in the navigational target must include one derived attribute (Rule 11). Derived attributes are attributes whose values are generated from other attributes using calculations, algorithms or procedures. Thus, we assume that the difference between EQ and EO is that an EQ does not contain derived or calculated data.

4.2. The numerical assignment rules

The numerical assignment rules help to produce a quantitative value that represents the Web application functional size. This is accomplished by applying two sets of rules that are introduced in the following subsections.

4.2.1. The OO-HFP measurement rules

These rules are used to identify the elements that add to the complexity of the identified functions (e.g., DETs, RETs, FTRs). According to the FPA metamodel, the complexity of a class (ILF) or "legacy" class (EIF) is determined by counting the number of data element types (DET) and the number of record element types (RET). The measurement rules proposed to identify the DETs and RETs of a class are shown in Appendix A, Table A-1. A DET is identified for each single attribute of a class, and a RET is identified for the class itself. In addition, we consider both aggregations/compositions and generalization/specialization relationships as contributing to the complexity of a class. This is because the FPA method suggests counting one DET for each piece of data that indicates a relationship with another ILF or EIF.

Associations, aggregations, and compositions are measured according to the multiplicity property of the relationship. This property specifies the lower/upper number of objects that can be associated to a single object of the class. If the number of instances in the target relationship is one, one DET is counted. Otherwise, if the number of instances in the target relationship is greater than one, one RET is identified. Specialization relationships are measured in the subclasses by counting one DET for each super class. Finally, the complexity of aggregation, composition, or inheritance hierarchies is measured by counting one RET for each class in the hierarchy. The complexity of a method (EI) or navigational target (EO and EQ) is determined by counting the number of data element types (DET) and file types referenced (FTR).

Some representative measurement rules for the complexity of a method of a class are shown in Table A-2. Each single argument of a method is counted as one DET, and each object-valued argument is counted as one FTR. Two DETs are counted in order to be compliant to FPA: one DET for the capability of the application to send a response message outside the boundary (error, confirmation, control) and another DET for the ability to specify an action to be taken. One FTR is also counted for the class where the method is declared. Additional FTRs are also counted for each one of the distinct classes that appear in the OCL formula (e.g., precondition, postcondition) of a method of a class.

Finally, some representative measurement rules for the complexity of a navigational target are shown in Table A-3. A navigational target groups elements (navigational classes, navigational links, and collections) to fulfill a user navigational requirement. One DET is counted for each element in a collection that implies navigating to another collection.

Additional DETs are counted for each visible attribute or method of a navigational class. One DET is counted for each link between navigational classes. This is because FPA suggests counting one DET for each piece of data that indicates a relationship with another group of data. Two other DETs are counted to be compliant to FPA: one DET for the capability of the application to send a response message and another for the ability to specify an action to be taken.

One FTR is counted for each navigational class of the navigational target. Additional FTRs are counted for the classes that appear in the OCL formula of a navigational class or navigational link (e.g., precondition, filter). There are also other measurement rules for navigational patterns (e.g., index, guided tour) that can be specified in a navigational link. The complete set of rules is described in the OO-H measurement guide [44].

4.2.2. The standard FPA counting rules

These rules are used to rate the complexity of the data and transactional functions, to assign weights to these functions, and finally, to aggregate the assigned values into an overall functional size value for the Web application. The ISO/IEC 20926 standard [25] provides several tables to determine the complexity levels of a function type. As an example, Table 3 shows the table to determine the complexity of an EO depending on the number of DETs and FTRs.

Then, a value in function points (FP) is assigned to each function depending on its type and complexity level. FPA also provides several tables to assign FP values for the identified functions. The proposed values for each function type are as follows:

- ILF: low = 7 FP, average = 10 FP, and high = 15 FP.
- EIF: low = 5 FP, average = 7 FP, and high = 10 FP.
- EI/EQ: 3 FP, average = 4 FP, and high = 6 FP.
- EO: low = 4 FP, average = 5 FP, and high = 7 FP.

Table 3
Complexity weights for EO (navigational target).

FTRs	1–5 DETs	6–19 DETs	20 or more DETs
0–1	Low	Low	Average
2–3	Low	Average	High
4 or more	Average	High	High

The function points assigned to each ILF (class or class hierarchy) and EIF (legacy class) are summed to obtain the functional size for the data functions (OO-HFP$_D$). The function points assigned to each EI (method of a class), EO, and EQ (navigational target) are summed to obtain the functional size for the transactional functions (OO-HFP$_T$). Finally, the values are summed to produce the functional size of the Web application in unadjusted function points: OO-HFP = OO-HFP$_D$ + OO-HFP$_T$.

5. Applying OO-HFP

Fig. 1 contains a representation of the procedure that is used to apply OO-HFP showing the relationships among OO-H, OO-HFP, and FPA. The Web size measurement process is represented according to the generalized representation of functional size measurement proposed by Fetcke et al. [18] (see Fig. 1b). This representation was applied to represent several FSM methods such as IFPUG FPA, Mark II FPA, and full function points [18]. According to this model, functional size measurement requires two steps of abstraction: identification and measurement.

OO-HFP conforms to this generalized representation of functional size measurement. During the *identification step* (using as input the OO-H conceptual model), an abstraction of the modeled Web application is made using the OO-HFP mapping rules. This abstraction contains only those elements of the application that are relevant to functional size measurement, according to the FPA metamodel. Next, during the *measurement step*, the OO-HFP measurement rules are used as part of the process of assigning numbers to the elements of the abstract model.

For each step of the OO-HFP measurement procedure, we also show the equivalent step in the process model proposed by Jacquet and Abran [28] (see Fig. 1b). According to this model, three steps are suggested for an application of the measurement method: software documentation gathering, construction of the software model, and application of the numerical assignment rules. In the remainder of this section, we follow this process model to illustrate the use of the OO-HFP measurement procedure in a case study.

5.1. Software documentation gathering

The documentation used to apply OO-HFP included: an OO-H conceptual model specification (class and navigational models) of a Web application for task management, and the OO-HFP measurement guide, which explains how to apply the procedure.

The task management application was developed for a Spanish Web development company. In this company, a Web development project involves a number of *Tasks* that are assigned to a *User* (a programmer of the company). Every task records the start date, the estimated end date, the priority, and so on. The project manager organizes the tasks in *Folders* according to some criteria: pending tasks, critical tasks, etc. Additionally, external *Files* can be associated to a task (e.g., requirement documents, code). The programmers can also record *Comments* to the tasks and send *Messages* to other programmers. Every working day, programmers can generate a report (*Daily-Report*) including the information related to the tasks that they were working on. Finally, the stakeholders of the project are recorded as *Contacts* in the application. Fig. 3 shows the UML class diagram with the most important information for this application.

Fig. 4a shows the navigational access diagram (NAD) level 0 for the *User*. The *Entry point User* requirement link indicates the entry point to the application. The first abstract page corresponds to a form where the user will log into the system. This process involves a *profile*, *user_name*, and *password* from the *Customers* navigational class. If the user exists (condition that is reflected by the LI4 navigational link), then a menu with a link to each one of the four identified navigational targets will be shown (i.e., the UML packages). This menu is represented by the collection construct *restricted home*. Each navigational target can be further "expanded" to show detailed internal navigation.

In OO-H, a navigational target may derive one or more *abstract pages*, depending on the value of the property 'effect' of a navigational link. The value *source* (a light arrow) indicates that the information specified in the navigational target will be presented in the current abstract page; the value *target* (a dark arrow) indicates that the information will be presented in a different abstract page. An analysis of this diagram shows that three different abstract pages will be generated: a page for the *home*, a page to show information about the authenticated user (*customers*), and a page for the user restricted menu (*restricted home*).

Fig. 4b provides detailed information about the *Contacts* navigational target. In this context, the *User* can see the information about *allContacts* or s/he can search for a given contact by providing an *initial* or by providing a *string*. These functionalities are represented by the three navigational links that connect the *contact menu* collection and the *Contact* navigational class. In addition, the user can create a new user by executing the method *New* in the *Contact1* navigational class. An analysis of this diagram shows that three different abstract pages will be generated: a page for the *contact menu*, a page to show information about contacts (*Contact*), and a page for the execution of the *New*( ) method (*Contact1*).

Similarly, Fig. 5 shows the details of the *Reports* navigational target. In this diagram, in the menu represented by the collection construct *Reports*, the user can see the reports *byContent* or *byDates* in the *Daily_Report1* navigational class.

By selecting the *title_and_author*, the user can see the detailed report as indicated in the *Daily_Report2* navigational class. In addition, in the *Reports* menu, the user has the option to see all the reports (*Daily_Report4* navigational class), all the users (*User1* navigational class), or *today's* report (*Daily_Report3* navigational class). Then the user can jump to the *Daily_Report1*

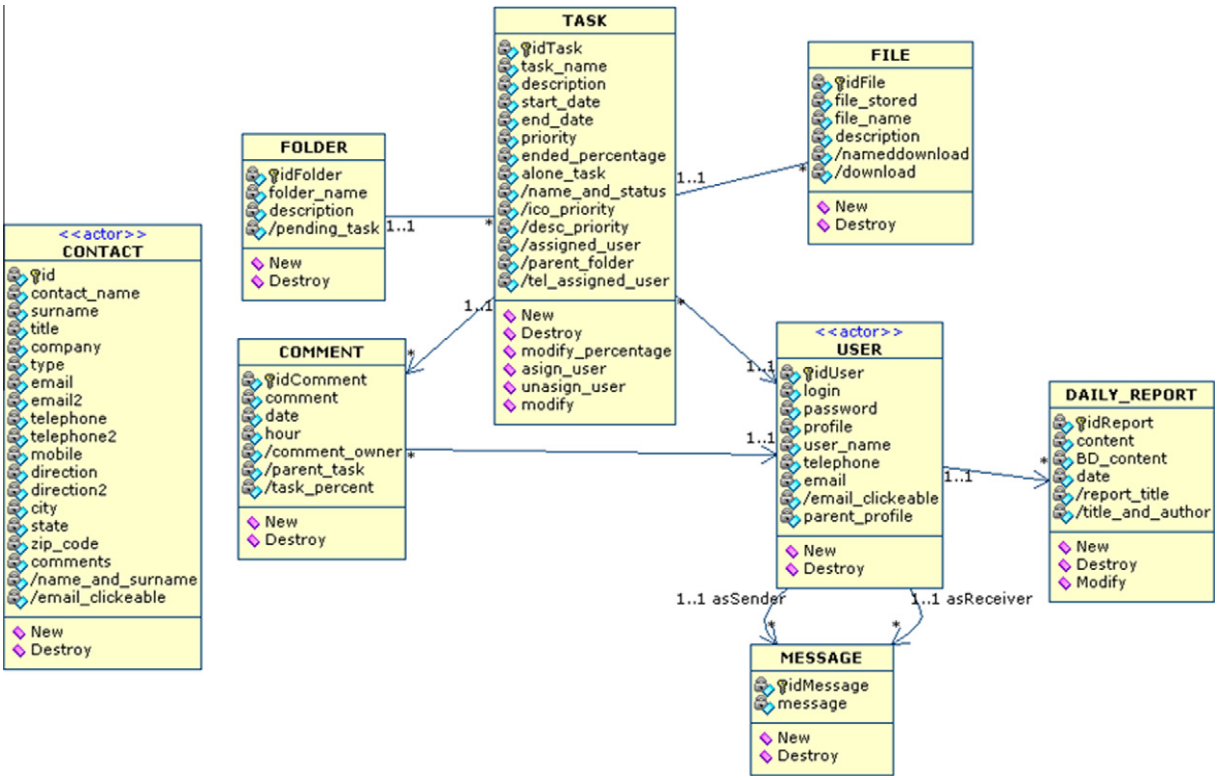

Fig. 3. UML Class Diagram for the task management Web application.

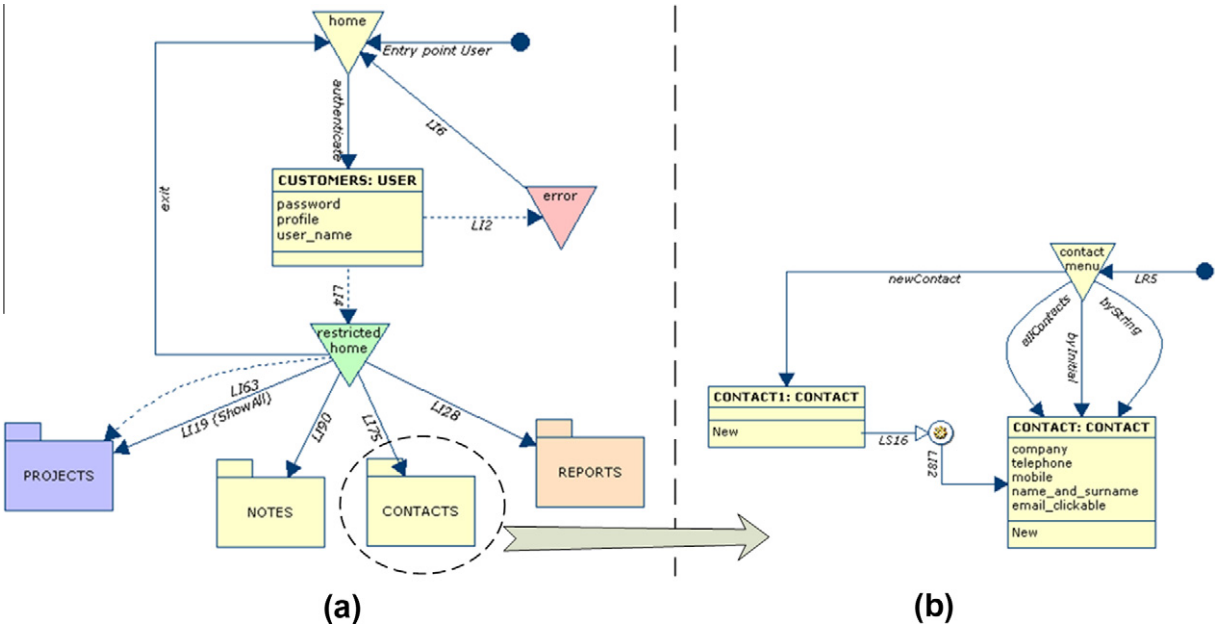

Fig. 4. Navigational Access Diagrams (NAD). (a) Shows the NAD level 0 for a User; (b) shows the Contacts navigational target.

navigational class (as shown in the LI47, LI73, and LI44 links). Finally, a new report can be created from the *Reports* menu by executing the method *New* in the *Daily_Report* navigational class. An analysis of this diagram shows that four different abstract pages will be generated: a page for the *Reports* collection, a page to execute the new( ) method (*Daily_Report*), another

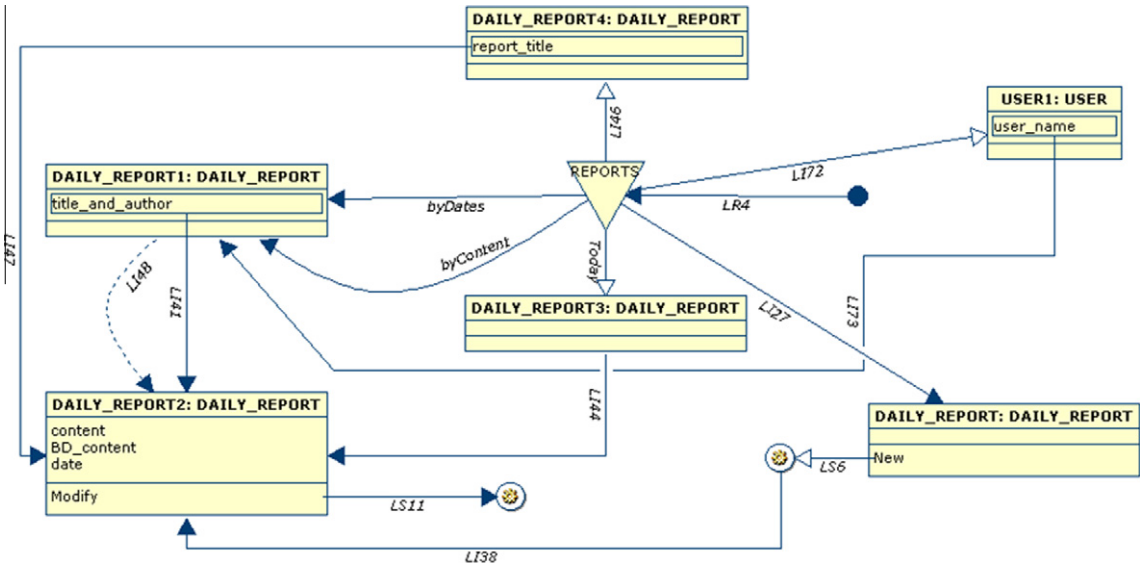

Fig. 5. Reports navigational target.

page to show the report containing the title and author (*Daily_Report1*), and a page to show the detailed report (*Daily_Report2*).

5.2. Construction of the software model

The software model for the task management application is built by applying the OO-HFP mapping rules. The counting scope includes the UML class diagram and the navigation access diagram (NAD) for this application (Rule 1). The application boundary is delineated by identifying the users and external applications that interact with it. The users of the application are indicated by the *User* and *Contact* classes since they have the "actor" stereotype (Rule 2). However, as these classes are also information that will be maintained by the application being built, they are included as part of the functionality of the application. The UML class diagram indicates that the application does not interact with any other external application since there is no class in the model with the "legacy" stereotype (Rule 3). As a consequence, all classes are considered to be inside the application boundary (Rule 4).

As previously explained, logical data files can be internal (ILFs) or external (EIFs), depending on whether they reside inside or outside the application boundary, respectively. By applying Rules 5–8, we identify the following candidate data functions for the task management application: *Comment*, *Contact*, *Daily-Report*, *File*, *Folder*, *Message*, *Task*, and *User*. All of them are classified as ILFs. No EIF has been identified.

The next step is to identify the transactional functions. According to Rule 9, an external input (EI) is identified for each method in the UML class diagram of the task management application (see Fig. 3). Therefore, 23 external inputs are identified. Finally, each navigational target of a NAD (and each abstract page derived from it) is identified as an EO or EQ, depending on whether or not they contain a derived attribute or method (Rules 10 and 11). In total, eight external inputs and 12 external inquiries have been identified.

For instance, for the *User* NAD level 0 (see Fig. 4a), three abstract pages have been identified. Two of them (*home* and *customers*) are classified as EQs since their purpose is to present data to the user; the other one (*restricted home*) is classified as EO since the navigational classes in these targets contain at least one derived attribute or method. Similarly, for the *Contact* navigational target (see Fig. 4b), three abstract pages have been identified. Two of them (*contact menu* and *Contact1*) are classified as EIs since their purpose is to present data to the user; the other one (*Contact*) is classified as EO since the *contact* navigational class has derived attributes (*name_and_surname* and *email_clickable*).

Finally, for the reports navigational context (see Fig. 5), four abstract pages have been identified. Two of them (*Daily_Report* and *Daily_Report2*) are classified as EQs since their purpose is to present data to the user; the other two (*Reports* and *Daily_Report1*) are classified as EOs since the *daily_report* navigational class contains at least one derived attribute.

5.3. Application of the numerical assignment rules

In order to obtain the functional size value for the Web application, we first apply the OO-HFP measurement rules and then the FPA counting rules.

5.3.1. Application of the OO-HFP measurement rules

Table A-4 shows the DETs and RETs that were counted for each class of the UML class diagram of the task management application when applying the measurement rules described in Table A-1. For instance, the *Message* class has four DETs and one RET. Two of the four DETs are due to the class attributes (Rule 12), and the other two are due to the association relationship with the *User* class (Rule 13). The RET is due to the *Message* class itself (Rule 17).

Table A-5 shows the DETs and FTRs for the methods of the classes of the task management application when applying the measurement rules described in Table A-2. Methods of classes are measured based on their signatures. For instance, the *New*( ) method of the *Contact* class has 21 DETs and one FTR. Nineteen DETs are due to the single arguments of the method (Rule 30) and the other two are due to the compliance of OO-HFP to the ISO/IEC 20926 standard (Rules 31 and 32). This standard suggests counting one DET for the system capacity to perform actions and another DET for the system capacity to send messages. The FTR is due to the class in which the method is declared (Rule 33).

Finally, Table A-6 shows the DETs and FTRs for the identified navigational targets of the task management application when applying the measurement rules described in Table A-3. For instance, the *Contact* navigational target (see Fig. 4b) has eight DETs and one FTR. Five of the DETs are due to the attributes of the *Contact* class (Rule 47), one is due to the *New* method in the class (Rule 48), and the remaining two are due to the compliance to the ISO/IEC 20926 standard (Rules 56 and 57).

5.3.2. Application of the FPA counting rules

Once the function elements (DETs, RETs and FTRs) have been identified, the tables provided in the ISO/IEC 20926 standard [25] are used to classify the data and transactional functions as having *low*, *average* or *high* complexity.

With respect to the data functions (classes) shown in Table A-4 (except for the *User* class) all of them are found to have a low complexity. The *User* class has an average complexity.

Table A-5 shows the number of DETs and FTRs for the identified methods of classes. In total, the task management application has 15 methods with low complexity, six methods with average complexity, and two methods with high complexity. For instance, the method *New* of the *Contact* class were found to have 21 DETs and one FTR. Thus, an average complexity is established for this method since FPA suggests that an EI with more than five DETs and two FTRs is in this complexity level.

Finally, Table A-6 shows the number of DETs and FTRs for the identified navigational targets and abstract pages of the task management application. From the eight identified EOs, three were classified as having low complexity, another three were classified as having average complexity, and two were classified as having high complexity. For instance, the *Contact* navigational target was found to have eight DETs and one FTR. By applying the complexity weights described in Table 3, a low complexity is established. With regard to the identified EOs, one was found to have high complexity and the remaining 11 were found to have low complexity.

The functional size is then obtained by mapping the function types and complexity ratings into numbers that represent the amount of functionality that these functions contribute to the functional size of the Web application. The complexity levels are attributed by applying several tables proposed by the ISO/IEC 20926 standard (see Section 4.2.2). As an example, the values assigned to an ILF are the following: low = 7 FP, average = 10 FP, and high = 15 FP. Note that the number of function points for each function appears in the last column of Tables A-4, A-5, A-6. Finally, these numbers are aggregated into an overall functional size value for the Web application.

To illustrate this process, the amount of functionality is first aggregated into values for the data and transactional functions. For the task management application, seven classes (ILFs) with low complexity and one class with average complexity are identified. When the weights suggested in the ISO/IEC 20926 standard (see Section 4.2.2) are applied, the obtained functional size for data functions is:

$$\begin{aligned} &\text{OO-HFP}_{\text{D}} = \text{OO-HFP}_{\text{ILF}} + \text{OO-HFP}_{\text{EIF}} \\ &\text{OO-HFP}_{\text{D}} = 59 + 0 = 59\ \text{FP} \end{aligned} \tag{1}$$

The detailed measurement for all transactional functions of the task management application is out of the scope of this paper; however, we have identified 20 methods (EIs) with low complexity, six methods with average complexity, and two methods with high complexity. In addition, eight navigational targets/abstract pages that represent EOs (three with low complexity, three with average complexity and two with high complexity) have been identified. Also, 13 navigational targets/abstract pages that represent EQs (11 with low complexity, and one with high complexity) were identified. By applying the weights suggested at the ISO/IEC 20926 standard (see Section 4.2.2), the obtained functional size for transactional functions is:

$$\begin{aligned} &\text{OO-HFP}_{\text{T}} = \text{OO-HFP}_{\text{EI}} + \text{OO-HFP}_{\text{EO}} + \text{OO-HFP}_{\text{EQ}} \\ &\text{OO-HFP}_{\text{T}} = 95 + 41 + 45 = 182\ \text{FP} \end{aligned} \tag{2}$$

Finally, the partial functional sizes obtained for data and transactional functions are summed to obtain the functional size value of the Web application. Thus, the functional size obtained for the task management Web application is 241 function points (FP).

6. The validation of OO-HFP for effort estimation

The validation of OO-HFP for Web effort estimation is done by comparing the prediction accuracy that it provides with the accuracy provided by the traditional FPA measure. The results presented in this section were obtained using the statistical software SPSS 15.0.1 for Windows.

6.1. Validation method

The validation of the prediction accuracy of the size measures is performed as follows [35,31,33]:

- Each of the two size measures is used as an independent variable to build an effort estimation model using a *simple Linear Regression*. Linear Regression [34] is a statistical technique that is used to build a prediction model (Equation). The equation represents the relationship between independent and dependent variables. We selected this technique because it provides accurate results when estimating Web projects (e.g., [49,37,15,17,35,19]). In our study, we predict the effort needed to develop a Web project based only on the knowledge of its size. Therefore, we employed *OO-HFP* and *FPA* as independent variables and *ActualEff* as a dependent variable.
- The *stability* of each effort estimation model is verified by following the procedure proposed by Mendes [35], which consists of the following steps:
  - The use of a residual plot showing residual vs. fitted values. The goal is to determine whether the residuals are random or normally distributed.
  - The use of Cook's distance values for all projects to identify influential data points. With this procedure, any projects that have distances greater than $3 \times (4/n)$ (where n represents the total number of projects) are removed from the data analysis. Those projects that have distances greater than $4/n$ but smaller than $(3 \times (4/n))$ are removed in order to test the model stability, which is done by observing the effect of their removal on the model. If the model coefficients remain stable and the R^2 improves, the influential projects are retained in the data analysis.
- The *effort predictions* of the final regression models obtained are measured using the mean magnitude of relative error (MMRE), median MRE (MdMRE), and Pred (0.25). These are commonly used measures for the evaluation of estimation models [12]. The magnitude of relative error (MRE) is calculated for each project whose effort is predicted:

$$\mathrm{MRE} = \left| \frac{\mathrm{Effort}_{\mathrm{ACTUAL}} - \mathrm{Effort}_{\mathrm{ESTIMATED}}}{\mathrm{Effort}_{\mathrm{ACTUAL}}} \right| \tag{3}$$

The prediction level (Pred) measure indicates a proportion of observations for a given level of accuracy. It is calculated as follows:

$$\mathrm{Pred}(l) = \frac{k}{N} \tag{4}$$

where, N is the total number of observations, and k is the number of observations with an MRE that is less than or equal to l. As in previous studies [49,37,17,19], we used 0.25 as a value for l. Consequently, the Pred (0.25) gives us the percentage of projects that were predicted with an MRE that is equal to or less than 0.25.

- As suggested by Kitchenham and Mendes [31], we also compared each model with predictions obtained using the mean and median efforts of the dataset in order to have a benchmark.
- Finally, we compared the accuracy between the obtained estimates using absolute residuals.

6.2. Results

The analysis presented in this paper is based on data from 31 Web projects provided by a Spanish Web development company. These projects were developed between 2003 and 2006. All the projects were developed using the OO-H method. The projects were typical Web applications, e.g., applications for cinema management, hospital management, Web-based content management, forums, intranets, etc. All the Web projects were new developments.

The sizes of the projects were measured using OO-HFP and FPA. The OO-HFP measure was obtained automatically using the VisualWADE [53] plug-in that implements this measurement procedure. The FPA size measure was obtained manually by a certified function point specialist using the IFPUG counting practices manual. The documentation that was used to calculate FPA included both the requirements specifications to develop the Web applications and the implemented applications.

Note that FPA is usually obtained using only the requirements specifications. However, previous studies[2] have shown that there exists a difference in size when the same project is measured at the requirements level and at the code level. Therefore, since our goal is to obtain a size measure for FPA that is as accurate as possible for comparison purposes, we applied this method

[2] http://www.devdaily.com/FunctionPoints/node32.shtml.

at the implemented Web application level. The effort estimation models were built using both measures by applying the data analysis procedure suggested by Mendes [35]. This procedure consists of five steps: data validation, variables and model selection, model construction, extraction of effort equation, and model validation.

6.2.1. Data validation

The purpose of this step is to understand the variables in order to help identify possible missing or unusual cases. Table 4 shows descriptive statistics for the variables in our study. The statistics for OO-HFP show that the functional size of the projects ranges from 24 FP to 2193 FP, whereas when FPA is applied, the size of the same projects ranges from 24 FP to 2001 FP. This indicates that the number of function points obtained by applying FPA is lower than the number of function points obtained by applying OO-HFP. One plausible reason could be that OO-HFP provides measurement rules for measuring the specific features of Web applications (e.g., links, data access structures).

This is in line with the results reported by Ruhe et al. [49] when Web objects and FPA were compared. These authors observed a difference of 55%, whereas we observed a difference of about 20%. This difference in size is due to the fact that Web objects adds four more functions (i.e., links, web building blocks, scripts, and multimedia files) to the five function types proposed in FPA, whereas OO-HFP considers only the five function types proposed in the standard (i.e., the new features of Web applications are measured in terms of rules for DETs, RETs, and FTRs).

The median for OO-HFP and *ActualEff* suggests that most of the projects were small with regard to their size and duration. Note that these applications were developed following a model-driven development approach where the source code was automatically generated using VisualWADE [53], hence, their short duration. The average development effort spent on a Web application (604 person-hours (ph)) is in line with other similar studies (883 ph) [49]. As reported by Ruhe et al. [49], the average development effort for Web applications is greater than the average development effort for Web hypermedia systems (between 58 and 154) [39]. In terms of dispersion, the collected data shows a higher dispersion (1109 ph) compared to the study of Ruhe et al. (710 ph).

In general, the number of function points for FPA is lower than the number of function points for OO-HFP. Although OO-HFP was designed to be compliant with FPA, it includes new measurement rules to take into account the specific features of Web applications (e.g., navigation and presentation). This could explain the difference in size observed. However, when comparing the number of function points (in FPA and OO-HFP) project by project, the size difference between the two measures is a maximum of 20%. This difference in size may have an impact on the effort estimation accuracy.

The number of staff on the development team (between 1 and 5) is in line with other studies published in the field of Web engineering [44,36]. These studies show that the typical size of a Web development team is small. In addition, the data indicated that the Web applications were mainly automatically generated using the following programming languages: Javascript (DHTML/DOM), PHP, HTML, SQL, and Python. Other projects also included the use of J2EE (Servlet, EJB). In terms of experience, the development team shows a median experience of 3.84 years in UML/OO-H. The modeling experience of the development team was ranked by the project manager on a scale of 1–5 (i.e., 1 = beginner, 2 = low, 3 = medium, 4 = advanced, 5 = expert). Finally, the experience of the development team in the programming languages used was between 4 and 8 months.

As a result of this preliminary analysis, we can conclude that none of the numerical values in our study seem to have unusual or missing values.

6.2.2. Variables and model selection

This step includes two phases: preliminary analysis and model building. The goal of the first phase is to choose the variables to use in the construction of an effort estimation model; the goal of the second phase is to construct the model itself.

The purpose of the preliminary analysis is to ensure that the variables comply with the assumptions of the effort estimation technique being used (i.e., regression analysis) [35,34]. These assumptions are as follows: (1) the independent variables are measured without error. If this does not apply, the variables should be normalized; (2) the relationship between dependent and independent variables is linear; (3) no important independent variables have been omitted; (4) the variance of the residuals is the same for all combinations of input variables; (5) the residuals must be normally distributed; (6) the residuals must not be correlated; (7) the independent variables are not linearly dependent.

In our study, each of the two size measures (i.e., OO-HFP, FPA) is used as an independent variable to build an effort estimation model using linear regression. The dependent variable is *ActualEff*. Thus, assumptions 3, 5, 6, and 7 do not apply in

Table 4
Descriptive statistics for the 31 Web projects.

Variable name	Variable description	*N*	Missing	Mean	Median	Std. Dev.	Min	Max
OO-HFP	Web application size using OO-HFP (FP)	31	0	379	181	508,02	24	2193
FPA	Web application size using FPA (FP)	31	0	345	169	457.80	24	2001
ActualEff	Development effort in person hours (ph)	31	0	604.0	98.00	1109.38	16	3644
DevTeam	Number of staff on the development team	31	0	3.23	3	1.521	1	5
NumLanguages	Number of programming languages	31	0	5.19	5	0.654	5	8
ExpProgLang	Experience of the development team in the programming languages (months)	31	0	5.61	6	1.453	4	8

our study since we have only one independent (input) variable to build each model. Assumption 4 was checked for compliance.

The next step is to look for symptoms that may suggest the need for variables to be normalized (e.g., skewness, outliers). To do this, we built a histogram and a boxplot for each one of the variables in our study. The histograms in Fig. 6 indicate that all the variables present skewed distributions (i.e., the distribution of values are asymmetrical).

Fig. 7 shows boxplots for the OO-HFP, FPA, and ActualEff variables. A boxplot is a tool that is suitable for checking the existence of outliers. The data shows the presence of outliers. We contacted the company to discuss the possible reasons for these large effort values for some projects. As a result, none of the obtained values seem to be invalid.

Therefore, the histograms and boxplots indicate symptoms of skewness and outliers. This indicates that the data should be normalized. The one-sample Kolmogorov–Smirnov test shown in Table 5 confirms this fact.

As suggested in [35], we used the natural log transformation to try to approximate the values to a normal distribution. To do this, we created three new variables *lOOHFP*, *lFPA*, and *lActualEff*, which contain the normalized values of the size in OO-HFP, FPA, and the actual effort. Finally, we used scatter plots to check the relationship between the independent (*lOOHFP* and *lFPA*) and dependent (*lActualEff*) variables. Fig. 8 shows that there is a strong relationship between *lActualEff* and *lOOHFP, and lActualEff* and *lFPA*, since the data points follow the line. The figure also shows that the relationship is positively associated since the values of size in OO-HFP and FPA tend to increase the actual effort.

6.2.3. Model construction

The regression-based effort model using the OO-HFP size measure, which was built using the 31 projects, presented an adjusted R^2 of 0.87; thus, it explains 87% of the variation in *ActualEff*. None of the projects presented distances greater than $3 \times (4/31)$. The P-P plot (Probability plot) shown in Fig. 9a suggests that the residuals are normally distributed. However, the residual plot for the 31 projects (see Fig. 9b) showed two projects that seemed to have large residuals. This was confirmed

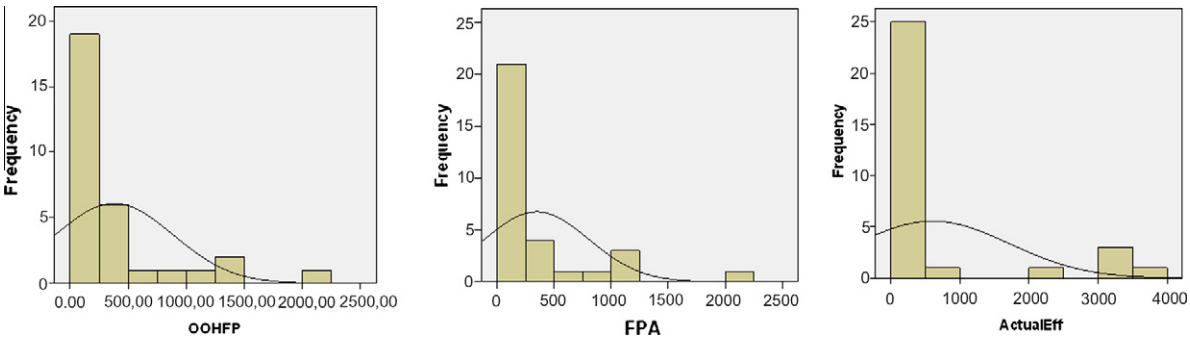

Fig. 6. Distribution of values for OO-HFP, FPA and ActualEff.

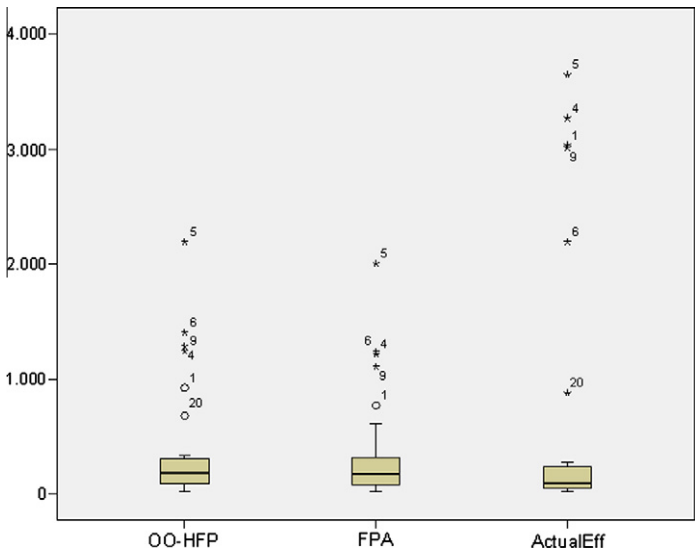

Fig. 7. Boxplots for the OO-HFP, FPA and ActualEff variables.

Table 5
One-tailed Kolmogorov–Smirnov test ($\alpha = 0.05$).

Independent variables	Mean	Std. Error	z	Sig.
OO-HFP	379.00	508.025	1.877	.0001
FPA	345.84	457.80	1.783	.0001
ActualEff	604.06	1109.38	2.346	.0000

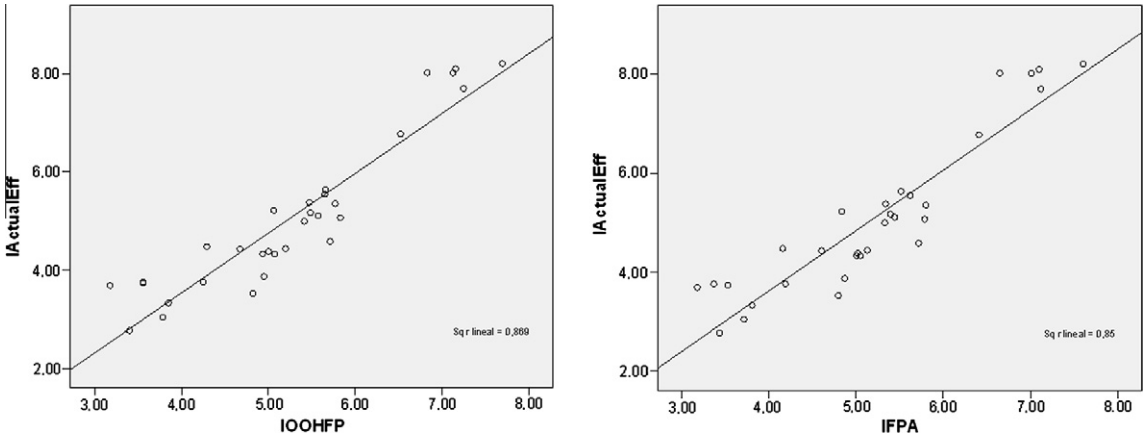

Fig. 8. Scatter plots showing relationships between *lActualEff* and *lOOHFP* and *lActualEff* and *lFPA*.

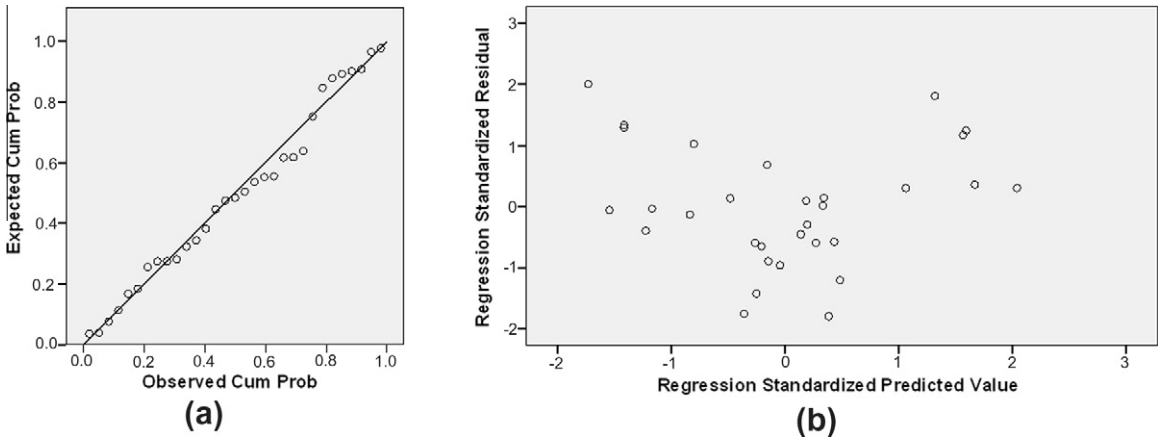

Fig. 9. PP-plot (a) and Residual plot (b) for the model obtained using OO-HFP.

using Cook's distance. Therefore, we removed these two projects from the dataset. To check the model's stability, a new model was generated without these two projects, giving an adjusted R^2 of 0.89. In the new model, the independent variables remained significant and the coefficients had similar values to those in the previous model. Therefore, the two high influence data points were not permanently removed from the dataset. Table 6 shows the coefficients for the regression-based model.

This gives the following equation:

$$lActualEff = -1.33 + 1.22 * \text{lOO-HFP} \tag{5}$$

Table 6
One-tailed regression-based model using OO-HFP ($\alpha = 0.05$).

Independent variables	Coefficient	Std. Error	Beta	t	Sig.
(constant)	−1.329	.471		−2.818	.004
OO-HFP	1.216	.088	.932	13.880	.000

which, when transformed back to the raw data scale, gives the final equation:

$$ActualEff = 0.264\ \text{OO-HFP}^{1.22} \tag{6}$$

The regression-based effort model using the FPA size measure, which is based on the 31 projects, presented an adjusted R^2 of 0.86. The P-P plot and the residual plot are presented in Fig. 10a and b, respectively. The residual plot for the 31 projects showed three projects that seemed to have very large residuals. To check the model's stability, a new model was generated without these projects, giving an adjusted R^2 of 0.88. This is slightly greater than the one for the previous model. In the new model, the independent variables remained significant and the coefficients had very similar values to those in the previous model, indicating that the high influence data points did not need to be permanently removed from further analysis. Table 7 shows the coefficients for the regression-based model.

This gives the following equation:

$$lActualEff = -1.26 + 1.22 * \text{lFPA} \tag{7}$$

which, when transformed back to the raw data scale, gives the final equation:

$$ActualEff = 0.283\ \text{FPA}^{1.22} \tag{8}$$

6.2.4. Model validation

The accuracy of the effort estimates was assessed using a *leave-one-out cross-validation* procedure. Cross-validation is the splitting of a dataset into training sets, which are used to build the effort estimation models, and validation sets, which are used to validate the models. In a leave-one-out cross validation, the original dataset is divided into n different subsets (n is the size of the dataset) of training and validation sets, where each validation set has only one project.

The cross-validation we employed was, therefore, a 31-fold cross-validation. This means that 31 training and validation sets were used to obtain accuracy statistics. Each time, we used 30 projects in the training set and 1 project in the validation set. The prediction accuracy of each model was measured using the Mean MRE (MMRE), Median MRE (MdMRE), and Pred (25) statistics. We also compared each model with predictions obtained using the mean and median efforts of the dataset in order to have a benchmark. Finally, as suggested by Maxwell [34], the statistical significance of the absolute residuals for OO-HFP and FPA was checked using the Wilcoxon Signed Ranks test, with $\alpha = 0.05$.

6.2.4.1. Comparison using summary statistics. The results of the cross-validation for OO-HFP are presented in Table 8. This table shows the measures of prediction accuracy that were calculated using the validation sets. It also shows the overall prediction accuracy that was obtained by aggregating the intermediate results for each model. The overall MMRE for the effort estimation model was 0.10, suggesting a good prediction accuracy. The MdMRE of 0.08 means that half of the projects in the validation sets presented MREs that were less than or equal to 0.08, and the other half presented MREs that were greater than or equal to 0.08. Based on the MdMRE, the overall effort prediction accuracy for the model is also good. Only two projects (Projects 7 and 14) had a MRE greater than 0.25. With regard to Pred (25), the results show that 93.5% of the projects in

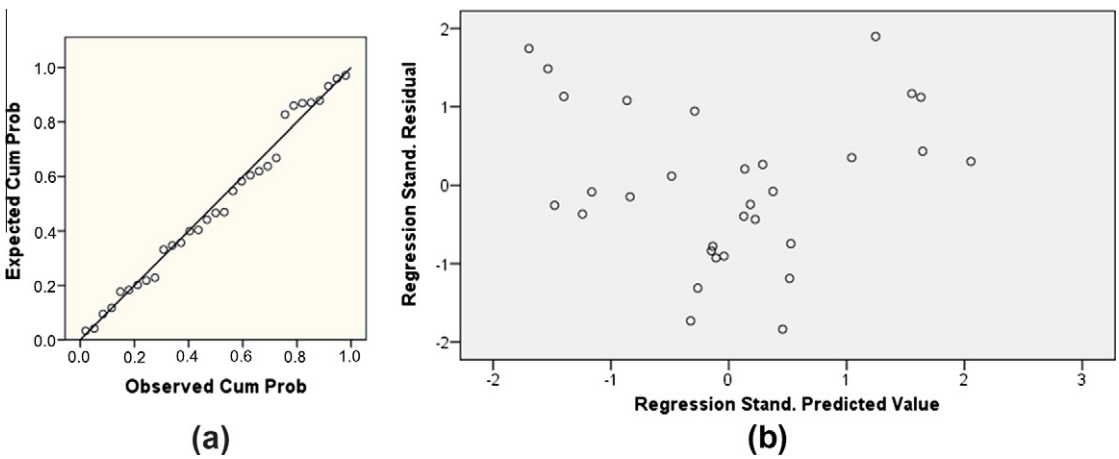

Fig. 10. PP-plot (a) and Residual plot (b) for the model obtained using FPA.

Table 7
One-tailed regression-based model using FPA ($\alpha = 0.05$).

Independent variables	Coefficient	Std. Error	Beta	t	Sig.
(constant)	−1.260	.505		−2.495	.0095
FPA	1.220	.095	.922	12.814	.0000

Table 8
Aggregated prediction accuracy statistics for Models 1–31 (OO-HFP).

	Project ID	Total effort (lActualEff)	Estimated effort	MRE	MRE <=0.25?	Absolute residuals
Set 1	1	8.02	6.85	0.15	TRUE	1.17
Set 2	2	4.48	3.86	0.14	TRUE	0.62
Set 3	3	3.33	3.38	0.01	TRUE	0.05
Set 4	4	8.09	7.28	0.10	TRUE	0.81
Set 5	5	8.2	7.96	0.03	TRUE	0.24
Set 6	6	7.69	7.49	0.03	TRUE	0.20
Set 7	7	3.69	2.36	0.36	FALSE	1.33
Set 8	8	2.77	2.79	0.01	TRUE	0.02
Set 9	9	8.01	7.23	0.10	TRUE	0.78
Set 10	10	5.17	5.36	0.04	TRUE	0.19
Set 11	11	4.33	4.67	0.08	TRUE	0.34
Set 12	12	4.38	4.75	0.08	TRUE	0.37
Set 13	13	5.11	4.90	0.04	TRUE	0.21
Set 14	14	3.53	4.59	0.30	FALSE	1.06
Set 15	15	5.07	5.81	0.15	TRUE	0.74
Set 16	16	5	5.28	0.06	TRUE	0.28
Set 17	17	5.55	5.56	0.00	TRUE	0.01
Set 18	18	5.36	5.70	0.06	TRUE	0.34
Set 19	19	5.63	5.58	0.01	TRUE	0.05
Set 20	20	6.78	6.58	0.03	TRUE	0.20
Set 21	21	3.74	2.92	0.22	TRUE	0.82
Set 22	22	3.04	3.29	0.08	TRUE	0.25
Set 23	23	3.76	3.82	0.02	TRUE	0.06
Set 24	24	5.38	5.34	0.01	TRUE	0.04
Set 25	25	5.22	4.82	0.08	TRUE	0.40
Set 26	26	4.44	5.03	0.13	TRUE	0.59
Set 27	27	3.87	4.72	0.22	TRUE	0.85
Set 28	28	4.33	4.85	0.12	TRUE	0.52
Set 29	29	4.43	4.36	0.02	TRUE	0.07
Set 30	30	4.58	5.67	0.24	TRUE	1.09
Set 31	31	3.76	2.92	0.22	TRUE	0.84
				MMRE = 0.10	Pred (25) = 93.5%	
				MdMRE = 0.08		

the validation sets presented estimated effort within 25% of the actual effort, which is a good result. This suggests that using the current effort model, a Web company has a 93.5% chance of obtaining an effort estimate for a project that is within 25% of its actual effort. Table 8 also shows the absolute residuals for each one of the validation sets. Absolute residuals in the context of this work mean the difference between the actual and the estimated effort.

The results of the cross-validation for FPA are presented in Table 9. The overall MMRE and MdMRE for the effort estimation model was 0.11 and 0.10, respectively. These results also suggest a good prediction accuracy. In terms of Pred (25), the results show that 87.1% of the projects in the validation sets presented estimated effort within 25% of the actual effort. Four projects were found to have a MRE greater than 0.25.

Overall, the results show that both OO-HFP and FPA presented good predictions if we assume as a reasonable threshold Conte et al.'s suggestion that good predictions should present a MMRE and MdMRE no greater than 25% and Pred (25) greater than or equal to 75% [12]. However, predictions obtained using OO-HFP presented a slightly better accuracy than FPA.

We also tested whether the MREs obtained for both measures are significantly different. Since the distribution of the data was not normal, we used the Wilcoxon Signed Ranks test (with $\alpha = 0.05$) to test for differences. The result obtained ($Z = -2125$, $p = 0.034$) allowed us to confirm that there is a significant difference between the MREs obtained for both measures. In particular, 15 observations had MREs for FPA greater than those for OO-HFP, 8 observations had MREs for FPA lower than those for OO-HFP, and 8 observations had nearly the same MREs for both measures.

In addition, we compared each model with predictions obtained using the mean and median efforts of the training sets in order to have a benchmark. The results shown in Table 10 indicate that OO-HFP provides better predictions than the ones based on the median or mean of the dataset.

6.2.4.2. Comparison using boxplots. It is also important to examine the absolute residuals of the aggregated models since a good prediction accuracy is directly related to how good the residuals are. The smaller the residuals, the better the indication that estimated and actual effort are similar. Fig. 11 shows the boxplots for the absolute residuals of the aggregate model for OO-HFP and FPA. The boxplots confirm the results obtained with the MMRE, MdMRE, and Pred (25) statistics. Specifically, they show that the spread of the distributions for OO-HFP are slightly wider than those for FPA. However, if we look at the medians, they indicate that FPA presents the largest residuals. In addition, FPA presents a slightly larger box when compared to OO-HFP. Therefore, according to the boxplots, the best predictions were obtained for OO-HFP, which were also confirmed by MMRE, MdMRE, and Pred (25) statistics.

Table 9
Aggregated prediction accuracy statistics for Models 1–31 (FPA).

	Project ID	Total effort (lActualEff)	Estimated effort	MRE	MRE<=0.25?	Absolute residuals
Set 1	1	8.02	6.78	0.15	TRUE	1.24
Set 2	2	4.48	3.79	0.15	TRUE	0.69
Set 3	3	3.33	3.40	0.02	TRUE	0.07
Set 4	4	8.09	7.28	0.10	TRUE	0.81
Set 5	5	8.2	8.00	0.02	TRUE	0.20
Set 6	6	7.69	7.42	0.04	TRUE	0.27
Set 7	7	3.69	2.47	0.33	FALSE	1.22
Set 8	8	2.77	2.93	0.06	TRUE	0.16
Set 9	9	8.01	7.17	0.10	TRUE	0.84
Set 10	10	5.17	5.32	0.03	TRUE	0.15
Set 11	11	4.33	4.93	0.14	TRUE	0.60
Set 12	12	4.38	4.89	0.12	TRUE	0.51
Set 13	13	5.11	4.97	0.03	TRUE	0.14
Set 14	14	3.53	4.57	0.29	FALSE	1.04
Set 15	15	5.07	5.83	0.15	TRUE	0.76
Set 16	16	5	5.24	0.05	TRUE	0.24
Set 17	17	5.55	5.60	0.01	TRUE	0.05
Set 18	18	5.36	5.82	0.09	TRUE	0.46
Set 19	19	5.63	5.47	0.03	TRUE	0.16
Set 20	20	6.78	6.53	0.04	TRUE	0.25
Set 21	21	3.74	2.97	0.21	TRUE	0.77
Set 22	22	3.04	3.28	0.08	TRUE	0.24
Set 23	23	3.76	3.86	0.03	TRUE	0.10
Set 24	24	5.38	5.25	0.02	TRUE	0.13
Set 25	25	5.22	4.60	0.12	TRUE	0.62
Set 26	26	4.44	5.02	0.13	TRUE	0.58
Set 27	27	3.87	4.69	0.21	TRUE	0.82
Set 28	28	4.33	4.87	0.12	TRUE	0.54
Set 29	29	4.43	4.35	0.02	TRUE	0.08
Set 30	30	4.58	5.79	0.26	FALSE	1.21
Set 31	31	3.76	2.73	0.27	FALSE	1.03
				MMRE = 0.11 MdMRE = 0.10	Pred (25) = 87.1%	

Table 10
Accuracy statistics for FPA and OO-HFP size measures.

Size measures	Prediction accuracy measures		
	MMRE	MdMRE	Pred (0.25)
OO-HFP	0.10	0.08	0.93
FPA	0.11	0.10	0.87
Mean effort	0.37	0.28	0.49
Median effort	0.35	0.23	0.62

When comparing the absolute residuals of the two samples, the Wilcoxon test revealed that OO-HFP presented better predictions than FPA ($z = -2.605$, p-value = 0.009). This means that the null hypothesis can be rejected and that the differences between the two samples of absolute residuals are significant.

In summary, what these results suggest is that, at least for the dataset employed, using effort estimates based on early Web size measures (OO-HFP) provides better accuracy than using estimates obtained with the FPA standard functional size method and better predictions than the mean or median of past projects.

6.2.4.3. Threats to validity. In terms of the internal validity of our results, the FPA size measure was obtained manually by a certified function point specialist using the IFPUG counting manual. The OO-HFP measure was well-documented with a measurement procedure, and the OO-HFP size measures were obtained automatically using the VisualWADE [53] plug-in that implements this measurement procedure.

One factor that could affect the results of our study is the size of the dataset employed (31 Web projects). However, we believe that this does not affect the validity of our results since we obtained significant statistical results. In addition, other studies for effort estimation conducted in the field of Web engineering also obtained significant results with smaller datasets of 12 and 15 Web projects [49,17,19].

In terms of the external validity, we employed a dataset that does not represent a random sample of Web projects. Consequently, our results can only be generalized to Web development companies that follow a model-driven Web development process that is based on the OO-H method. Nevertheless, it can be considered a representative method of the whole set of

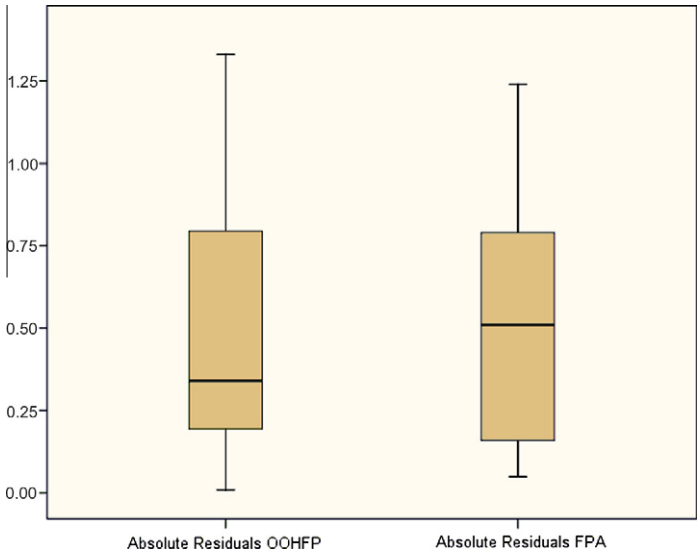

Fig. 11. Bloxplots for the absolute residuals of the agregate model for OO-HFP and FPA.

model-driven Web development methods [40]. The equivalence of the primitives of this method with respect to other model-driven Web development methods (e.g., UWE, WebML) is described in [8]. By applying these guidelines, OO-HFP can easily be adapted for sizing Web projects that were developed using other model-driven Web development methods.

The fact that the full sample comes from a single company somewhat decreases the validity of our results. Although some studies have suggested that within-company datasets are needed to produce accurate effort estimates [29,30], we are aware that the time required to accumulate enough data on past projects may be prohibitive for certain companies. Thus, to increase external validity, the current study needs to be replicated using data collected from different Web companies.

7. Conclusions and further work

This paper has presented the validation of a functional size measure (OO-HFP) for effort estimation in a model-driven Web development process. This measure is compliant to FPA, which is a widely used FSM method in industry. The purpose of the validation was to compare the prediction accuracy that OO-HFP provides with the accuracy provided by the standard FPA method.

The results showed (at least for the dataset employed) that the effort estimates obtained for projects that are sized using OO-HFP is more accurate than the effort estimates obtained using the standard FPA method. The results also showed that OO-HFP provides better predictions than the ones based on the median or mean of the dataset. This indicates that by following a model-driven development approach according to the OO-H development process, the size measure obtained at the conceptual model of a Web application can be considered a suitable effort predictor. This is in line with our previous results [2], which were obtained by comparing OO-HFP with a general-purpose Web size measure (i.e., the Tukutuku measures [37]). Thus, our results provide more evidence about the usefulness of a model-based Web size measure for effort estimation purposes. Since models are considered to be the most important artifacts in model-driven development, instead of predicting the amount of effort needed to develop a Web application based on the source code, we predicted it based on the corresponding conceptual models.

As future work, we plan to conduct further investigations using larger datasets of Web projects collected from culturally and organizationally different Web companies to ensure the external validity. We also plan to replicate this study using other effort estimation techniques (e.g., case-based reasoning) and other size measures (e.g., Web objects). Since the OO-HFP was tailored to fit the OO-H method, which was defined following the MDA standards for model-driven development (e.g., MOF, UML), many of the modeling primitives in the OO-H metamodel can also be found in other Web development methods [7,10,50]. Therefore, we plan to investigate to what extent OO-HFP can be adapted for use in other model-driven Web development approaches.

Acknowledgment

This research work is supported by the "Ministerio de Ciencia e Innovación" under the MULTIPLE project (with ref. TIN2009-13838), Spain.

Appendix A. Measurement rules

Tables A-1, A-2, A-3 describe some representative OO-HFP measurement rules for class, class hierarchies, method and navigational target. Tables A-4, A-5, A-6 show the results of applying these rules to the case study described in Section 5.

Table A-1
Measurement rules for the complexity of a class and class hierarchies.

Rules for DET	Rules for RET
Rule 12. Count 1 DET for each single attribute type of the class (i.e., String, Integer)	Rule 17. Count 1 RET for the class
Rule 13. Count 1 DET for each association with a class or "legacy" class having a maximum multiplicity of 1	Rule 18 Count 1 RET for each association relationship with a class or "legacy" class having a maximum multiplicity of *
Rule 14. Count 1 DET for each aggregation with a class having a maximum multiplicity of 1	Rule 19. Count 1 RET for each aggregation relationship with a class having a maximum multiplicity of *
Rule 15. Count 1 DET for each composition with a class having a maximum multiplicity of 1	Rule 20. Count 1 RET for each composition relationship with a class having a maximum multiplicity of *
Rule 16. Count 1 DET for each super class in an inheritance relationship	Rule 21. Count 1 RET for each class in an aggregation, composition or inheritance hierarchy

Table A-2
Some representative measurement rules for the complexity of a method of a class.

Rules for DET	Rules for FTR
Rule 30. Count 1 DET for each single argument of the method	Rule 33. Count 1 FTR for the class in which the method is declared
Rule 31. Count 1 DET for the capability of the application to send a message outside the boundary	Rule 34. Count 1 FTR for each complex argument (object of a class[a])
Rule 32. Count 1 DET for the ability of the application to specify an action to be taken	Rule 35. Count 1 FTR for each class[a] in the OCL formula of a precondition
	Rule 36. Count 1 FTR for each new class[a] in the OCL formula of a postcondition

[a] Only if the class has not been counted yet.

Table A-3
Some representative measurement rules for the complexity of a navigational target.

Rules for DETs	Rules for RETs
Rule 46. Count 1 DET for each element (link) of a collection that implies navigating to a collection	Rule 58. Count 1 FTR for each navigational class
Rule 47. Count 1 DET for each visible attribute of a navigational class	Rule 59. Count 1 FTR for each class[a] in the OCL formula of a filter defined upon navigational links
Rule 48. Count 1 DET for each method of a navigational class	Rule 60. Count 1 FTR for each class[a] in the OCL formula of a precondition defined upon links
Rule 50. Count 1 DET for each navigational link between two navigational classes	Rule 61. Count 1 FTR for each class[a] in the OCL formula of a navigational class
Rule 56. Count 1 DET for the capability of the application to send a message	
Rule 57. Count 1 DET for the ability of the application to specify an action to be taken	

[a] Only if the class has not been counted yet.

Table A-4
Measurement of classes.

Function description	DETs	RETs	Complexity	FPs
COMMENT	6	1	Low	7
CONTACT	17	1	Low	7
DAILY_REPORT	5	1	Low	7
FILE	5	1	Low	7
FOLDER	3	2	Low	7
MESSAGE	4	1	Low	7
TASK	10	3	Low	7
USER	8	6	Average	10

Table A-5
Measurement of methods of classes.

Function description	DETs	FTRs	Complexity	FPs
COMMENT.Destroy	2	1	Low	3
COMMENT.New	6	3	High	6
CONTACT.Destroy	2	1	Low	3
CONTACT.New	**21**	**1**	**Average**	**4**
DAILY_REPORT.Destroy	2	1	Low	3
DAILY_REPORT.Modify	6	1	Low	3
DAILY_REPORT.New	6	2	Average	4
ENUM_PRIORITY.Destroy	2	1	Low	3
ENUM_PRIORITY.New	5	1	Low	3
FILE.Destroy	2	1	Low	3
FILE.New	6	2	Average	4
FOLDER.Destroy	2	1	Low	3
FOLDER.New	5	1	Low	3
MESSAGE.Destroy	2	1	Low	3
MESSAGE.New	4	3	Average	4
TASK.asign_user	2	2	Low	3
TASK.Destroy	2	1	Low	3
TASK.modify	9	2	Average	4
TASK.modify_percentage	9	2	Average	4
TASK.New	9	4	High	6
TASK.unasign_user	2	2	Low	3
USER.Destroy	2	1	Low	3
USER.New	10	1	Low	3

Table A-6
Measurement of navigational targets.

Function type	Function description	DETs	FTRs	Complexity	FPs
External outputs	DETAIL_CLASSIFIER	17	3	Average	5
	COMMENT1	7	1	Low	4
	CONTACT	**8**	**1**	**Low**	**4**
	TASK_DETAIL	28	4	High	7
	Restricted home	21	1	Average	5
	REPORTS	32	3	High	7
	DAILY_REPORT1	10	1	Low	4
	TAREAS_PROPIAS	13	2	Average	5
External inquiries	CLASIFICADOR	59	3	High	6
	Customers	10	1	Low	3
	CONTACT1	3	1	Low	3
	Home	6	1	Low	3
	Home news	18	0	Low	3
	DAILY_REPORT	5	1	Low	3
	DAILY_REPORT2	6	1	Low	3
	Contact menu	18	1	Low	3
	NO INCIDENCES	2	0	Low	3
	WITHOUT TASKS	2	0	Low	3
	TASK1	5	2	Low	3
	USER	3	1	Low	3

References

[1] S. Abrahão, G. Poels, A family of experiments to evaluate a functional size measurement procedure for Web applications, Journal of Systems and Software 82 (2) (2009) 253–269.
[2] S. Abrahão, E. Mendes, J. Gomez, E. Insfran, A model-driven measurement procedure for sizing Web applications: design, automation and validation, in: Proceedings of the ACM/IEEE 10th International Conference on Model-Driven Engineering Languages and Systems (MODELS'07), Nashville, TN, USA, LNCS, vol. 4735, Springer-Verlag, 2007, pp. 467–481.
[3] S. Abrahão, G. Poels, O. Pastor, Evaluating a functional size measurement method for Web applications: an empirical analysis, in: Proceedings of the 10th IEEE Software Metrics Symposium (METRICS'04), USA, 2004, pp. 358–369.
[4] S. Abrahão, O. Pastor, Measuring the functional size of web applications, International Journal of Web Engineering and Technology 1 (1) (2003) 5–16.
[5] G. Antoniol, C. Lokan, G. Caldiera, R. Fiutem, A function point-like measure for object-oriented software, Empirical Software Engineering 4 (3) (1999) 263–287.
[6] L. Baresi, S. Morasca, Three empirical studies on estimating the design effort of Web applications, ACM Transaction on Software Engineering and Methodology 16 (4) (2007). article 15.
[7] L. Baresi. F. Garzotto, P. Paolini, From web sites to web applications: new issues for conceptual modeling, in: Proceedings of the ER'2000 Workshop on Conceptual Modeling and the Web, 2000, pp. 89–100.

[8] C. Cachero, S. Melia, M. Genero, G. Poels, C. Calero, Towards improving the navigability of Web applications: a model-driven approach, European Journal of Information Systems 16 (2007) 420–447.
[9] E.J.D. Cândido, R. Sanchez, Estimating the size of web applications by using a simplified function point method, in: Proceedings of the WebMedia/LA-WEB 2004, 2004, pp. 98–105.
[10] S. Ceri, P. Fraternali, A. Bongio, Web modeling language (WebML): a modeling language for designing Web sites, in: Proceedings of the 9th Word Wide Web Conference (WWW'00), Amsterdam, The Netherlands, 2000, pp. 137–157.
[11] D. Cleary, Web-based development and functional size measurement, in: Proceedings of the IFPUG Annual Conference, San Diego, USA, 2000.
[12] S.D. Conte, H.E. Dunsmore, V.Y. Shen, Software Engineering Metrics and Models, The Benjamin/Cummings Publishing Company, Inc., 1986.
[13] Cost Xpert, Cost Xpert Group Inc. Estimating internet development, 2001. <http://www.costxpert.com/Reviews_Articles/SoftDev/> (retrieved March 12.03.09).
[14] G. Costagliola, F. Ferrucci, G. Tortora, G. Vitiello, Class point: an approach for the size estimation of object-oriented systems, IEEE Transactions on Software Engineering 31 (1) (2005) 52–74.
[15] G. Costagliola, S. Di Martino, F. Ferrucci, C. Gravino, G. Tortora, G. Vitiello, A COSMIC-FFP approach to predict web application development effort, Journal of Web Engineering 5 (2) (2006) 93–120.
[16] Cutter Consortium, Poor project management – problem of e-projects, 2000, <http://www.cutter.com/consortium/press/001019.html> (retrieved 16.03.09).
[17] S. Di Martino, F. Ferrucci, C. Gravino, E. Mendes, Comparing size measures for predicting web application development effort: a case study, in: Proceedings of the International Symposium on Empirical Software Engineering and Measurement (ESEM'07), Madrid, Spain, 2007, pp. 324–333.
[18] T. Fetcke, A. Abran, R. Dumke, A generalized representation for selected functional size measurement methods, in: Proceedings of the 11th International Workshop on Software Measurement, Montréal, Canada, Shaker Verlag, 2001, pp. 1–25.
[19] F. Ferrucci, C. Gravino, S. Di Martino, A case study using Web objects and COSMIC for effort estimation of Web applications, in: Proceedings of the 34th Euromicro Conference on Software Engineering and Advanced Applications (SEAA'08), Parma, Italy, 2008, pp. 441–448.
[20] P. Fraternali, M. Tisi, A. Bongio, Automating function point analysis with model driven development, in: Proceedings of the Conference of the Centre for Advanced Studies on Collaborative Research (CASCON'06), Markham, Ontario, Canada, 2006, pp. 233–247.
[21] F. García, M.A. Serrano, J.A. Cruz-Lemus, F. Ruiz, M. Piattini, Managing software process measurement: a metamodel-based approach, Information Sciences 177 (12) (2007) 2570–2586.
[22] J. Gomez, C. Cachero, O. Pastor, Conceptual modeling of device-independent web applications, IEEE MultiMedia 8 (2) (2001) 26–39.
[23] IFPUG, Hints to counting web sites, IFPUG White Paper, 1998.
[24] International Software Benchmarking Standards Group (ISBSG), <http://www.isbsg.org> (retrieved 03.06.09).
[25] ISO/IEC 20926: Software engineering – IFPUG 4.1, Unadjusted functional size measurement method – counting practices manual, 2003.
[26] ISO/IEC 19761, Software engineering – COSMIC-FFP, A functional size measurement method, 2003.
[27] ISO, International Vocabulary of Basic and General Terms in Metrology, second ed., 1993.
[28] J.P. Jacquet, A. Abran, From software metrics to software measurement methods: a process model, in: Proceedings of the Third International Standard Symposium and Forum on Software Engineering Standards (ISESS'97), Walnut Creek, USA, 1997, pp. 128–135.
[29] R. Jeffery, M. Ruhe, A. Wieczorek, A comparative study of two software development cost modeling techniques using multi-organizational and company-specific data, Information and Software Technology 42 (14) (2000) 1009–1016.
[30] C.F. Kemerer, An empirical validation of software cost estimation models, Communications of the ACM 30 (5) (1997) 416–429.
[31] B.A. Kitchenham, E. Mendes, A comparison of cross-company and single-company effort estimation models for web applications, in: Proceedings of the Eighth International Conference on Empirical Assessment in Software Engineering (EASE'04), Edinburgh, Scotland, 2004, pp. 47–55.
[32] L. Mangia, R. Paiano, MMWA: a software sizing model for web applications, in: Proceedings of the Fourth International Conference on Web Information Systems Engineering (WISE'03), Rome, Italy, 2003, pp. 53–61.
[33] K.D. Maxwell, What you need to know about statistics, in: E. Mendes, N. Mosley (Eds.), Web Engineering, Springer, 2005, pp. 365–408.
[34] K. Maxwell, Applied statistics for software managers, Software Quality Institute Series, Prentice–Hall, 2002.
[35] E. Mendes, Cost Estimation Techniques for Web Projects, IGI Publishing, 2007.
[36] E. Mendes, N. Mosley, S. Counsell, Investigating web size metrics for early web cost estimation, Journal of Systems and Software 77 (2) (2005) 157–172.
[37] E. Mendes, N. Mosley, S. Counsell, Investigating early web size measures for web cost estimation, in: Proceedings of the Seventh Conference on Evaluation and Assessment in Software Engineering (EASE'03), Keele, UK, 2003, pp. 1–22.
[38] E. Mendes, S. Counsell, N. Mosley, Web metrics – estimating design and authoring effort, IEEE Multimedia, Special Issue on Web Engineering, January–March, 2001, pp. 50–57.
[39] E. Mendes, N. Mosley, S. Counsell, A comparison of length, complexity and functionality as size measures for predicting Web design and authoring effort, in: Proceedings of the International Conference on Empirical Assessment in Software Engineering (EASE'01), Keele, UK, 2001, pp. 1–14.
[40] N. Moreno, A. Vallecillo, Towards interoperable Web engineering methods, Journal of the American Society for Information Science and Technology 59 (7) (2008) 1073–1092.
[41] S.F. Ochoa, M.C. Bastarrica, G. Parra, Estimating the development effort of web projects in chile, in: Proceedings of the 1st Latin American Web Congress (LA-WEB'03), Chile, 2003, pp. 114–124.
[42] OMG, UML 2.1 Unified Modeling Language™, 2006.
[43] OMG, MDA Guide, Version 1.0.1, 2003. <http://www.omg.org/docs/omg/03-06-01.pdf> (retrieved 12.03.09).
[44] OO-HFP Measurement Guide. <www.dsic.upv.es/~sabrahao/OO-HFP> (retrieved 02.04.09).
[45] O. Pastor, J. Fons, V. Pelechano, S. Abrahão, Conceptual modeling of Web applications: the OOWS approach, in: E. Mendes, N. Mosley (Eds.), Web Engineering, Springer, 2006, pp. 277–302.
[46] D. Reifer, Web development: estimating quick-to-market software, IEEE Software 17 (6) (2001) 57–64.
[47] T. Rollo, Sizing E-commerce, in: Proceedings of the Australian Conference on Software Metrics (ACOSM'00), Sydney, Australia, 2000.
[48] M. Ronchetti, G. Succi, W. Pedrycz, B. Russo, Early estimation of software size in object-oriented environments a case study in a CMM level 3 software firm, Information Sciences 176 (2006) 475–489.
[49] M. Ruhe, R. Jeffery, I. Wieczorek, Using web objects for estimating software development effort for web applications, in: Proceedings of the Ninth International Software Metrics Symposium, 2003, pp. 30–37.
[50] D. Schwabe, G. Rossi, The object-oriented hypermedia design model, Communications of the ACM 38 (8) (1995) 45–46.
[51] Total Metrics, Web based applications: how is FPA applied to sizing Web based applications, 2001, <http://www.totalmetrics.com/cms/servlet/main2?Subject=Content&ID=203> (retrieved 03.03.09).
[52] T. Uemura, S. Kusumoto, K. Inoue, Function point measurement tool for UML design specification, in: Fifth International Software Metrics Symposium (METRICS'99), Florida, USA, 1999.
[53] VisualWade, <www.visualwade.com> (retrieved 23.04.09).